Informe de práctica de la semana 6

Repositorios iteradores y medición del rendimiento

UNAP · Ingeniería de Sistemas · SIS206 POO II · 2026 II
S6-001 · 19 de octubre de 2026 · Informe revisado

1 Propósito y actividades

Completar y comprobar los repositorios, consultas, recorridos y mediciones presentes en la carpeta de la semana 6, adaptándolos también al hotel.

No se dispone de una guía oficial de esta semana. El alcance se deriva de actividad_02, repositorios, iteradores y benchmark existentes. Se identificó que eliminar devolvía False al borrar una entidad con valor falso; que la ventana admitía cero o negativos; y que el benchmark reconstruía dict y set durante cada búsqueda.

Actividades desarrolladas

Actividad	Archivo o componente	Resultado
Repositorio	python/repositorio.py y cpp/repositorio.hpp	Guardar o actualizar, insertar sin duplicados, buscar, eliminar y recorrer.
Consultas	python/solucion.py y cpp/solucion.hpp	Filtrar destacados, ordenar el ranking y obtener estadísticas.
Iteradores	Repositorio, ColaAtencion y PlanillaIterador	FIFO, recorrido independiente y copia segura del contenedor C++.
Ventanas	ventana_deslizante y ventanas	Tres ventanas válidas; rechazo de k fuera de 1 a longitud.
Benchmark y hotel	python/benchmark.py y cpp/benchmark.cpp	Construcción separada y tres consultas hoteleras.

Caso explicado

El repositorio almacena el valor 0 con identificador 0. La primera eliminación debe devolver True porque el registro existía, aunque su valor sea falso al convertirlo a booleano. La segunda devuelve False porque ya no está. Esta diferencia permite comprobar la existencia del registro sin confundirla con su contenido.

Conceptos para interpretar el caso

Un repositorio genérico reutiliza operaciones para distintos tipos de entidad. Un iterador mantiene la posición de un recorrido. En una medición de rendimiento, el tiempo de construir la estructura debe distinguirse del tiempo de consultarla.

La solución y sus pruebas se encuentran en desarrollo. Las respuestas completas y el notebook amplían los ejemplos de este informe.

2 Desarrollo en Python y C++

guardar conserva la semántica de actualizar del material original. insertar ofrece rechazo de duplicados de manera explícita. buscar_resultado distingue una búsqueda fallida incluso si el dato almacenado es None. eliminar comprueba la existencia del ID y no la verdad del valor borrado.

PlanillaIterador conserva una instantánea de los datos ordenados. C++ crea sus iteradores al comenzar el recorrido y no guarda iteradores vinculados a otra copia del vector. ColaAtencion usa deque para atender por orden de llegada.

El generador Python produce una ventana al consumirla. La versión C++17 devuelve todas las ventanas en un vector; su mayor consumo de memoria se declara. Las dos versiones validan k positivo y menor o igual al tamaño. El benchmark usa cuatro tamaños, 200 consultas iguales por lote y mediana de tres repeticiones, separando construcción y búsqueda.

Colecciones y contratos

Operación	Python	C++
Mapa por ID	dict de entidades	unordered_map con extractor de ID
Ausencia	None o Result fallido	optional sin valor o Result fallido
Filtrar	Predicado sobre cada entidad	Predicado sobre cada entidad
Ordenar	sorted con key y reverse	sort con comparador
FIFO	deque append y popleft	deque push_back y pop_front
Ventana	Generación incremental	Vector de ventanas materializadas

Cómo seguir la implementación

Repositorio[Entidad, ID] usa Generic en Python y Repositorio<T, ID> usa templates en C++. Ambos reciben cómo extraer el identificador. filtrar aplica una condición y ordenar construye una vista ordenada. La ventana Python se produce al recorrer el generador; C++17 reúne sus ventanas en un vector y utiliza más memoria para conservarlas.

3 Verificación y resultados

Las pruebas registradas aprobaron 15 casos Python y 14 casos C++17. Se ejecutaron en Python 3.12.14 y g++ 10.3.0. Las demostraciones de ambos lenguajes producen la misma salida.

Comprobación	Resultado	Registro
Pruebas Python	15 aprobadas	evidencias/pruebas_python.txt
Pruebas C++17	14 aprobadas	evidencias/pruebas_cpp.txt
Compilación y demostraciones	Sin advertencias y salidas iguales	evidencias/verificacion.json

Pytest ejecutó los mismos casos que unittest; sus resultados no se suman como pruebas diferentes. Las compilaciones usaron -Wall, -Wextra, -Wpedantic y -Werror.

Mediciones con un millón de elementos

Lenguaje y estructura	Construcción en ms	Búsqueda de lote en ms
Python list	33.6440	1389.2981
Python dict	82.6500	0.0178
Python set	57.0706	0.0155
Python bisect	24.3856	0.0913
C++ vector	2.2593	42.0091
C++ unordered_map	124.1530	0.0157
C++ unordered_set	104.8310	0.0101
C++ binary_search	1.1283	0.0222

Los tiempos de búsqueda comparan 200 consultas equivalentes por lote y usan la mediana de tres repeticiones. La tabla hash tiene búsqueda promedio esperada O(1), no una garantía para cualquier caso. Las cifras dependen del equipo, el contenedor, el tamaño y el entorno de ejecución.

C++ se compiló con -O2. Los 32 registros completos y los aciertos están en los CSV de evidencias. La construcción de una tabla hash puede costar más que preparar un vector, aunque sus búsquedas posteriores sean más rápidas en este ensayo.

Las pruebas funcionales también verifican eliminación del valor 0, rechazo de duplicados y tres ventanas de tamaño 3 sobre cinco datos. El promedio académico obtenido fue 14.125 y el promedio de precio nocturno del hotel fue S/ 142.50.

4 Aplicación al hotel y conclusiones

Repositorio<T,ID> o Repositorio[Entidad,ID] administra habitaciones por número, preservando la misma implementación para otras entidades. Los identificadores de las entidades del hotel son de solo lectura. Las colecciones devueltas son copias del contenedor; eso no implica clonar profundamente cada objeto compartido.

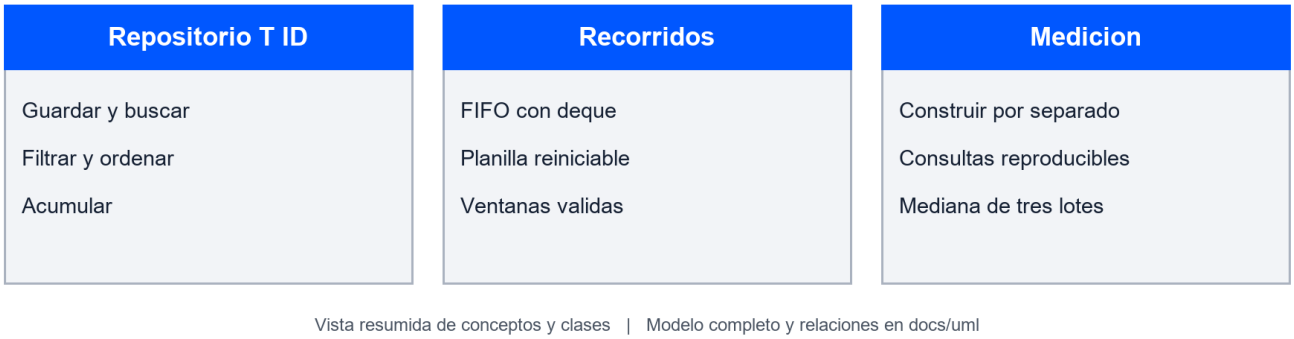


Figura 1 Esquema de los conceptos aplicados al hotel. El diagrama de clases completo se encuentra en docs/uml.

Conclusión de la práctica

El repositorio distingue actualización, duplicado y ausencia; los recorridos se repiten sin compartir posiciones inválidas. El benchmark separa preparación y consulta para comparar la operación estudiada. El alcance procede del código existente y queda pendiente contrastarlo con la guía oficial de la semana 6.

Para comprobar la comprensión, seguir el caso explicado en los dos lenguajes e identificar la validación, el cambio de estado y la prueba que lo verifica. El anexo ACTIVIDADES_RESUELTAS.md desarrolla las preguntas de reflexión.

Transparencia y datos de entrega

Se utilizó Codex para analizar el material, desarrollar y verificar el código y preparar los documentos. Antes de entregar, completar los datos del estudiante y GitHub. La revisión de pares y la sustentación requieren participación humana.

Referencias

Fowler, M. (2018). *Refactoring Improving the Design of Existing Code* (2.ª ed.). Addison Wesley. https://martinfowler.com/books/refactoring.html

Material proporcionado. (s. f.). *Códigos de la carpeta sem6*. Repositorios, iteradores y benchmark. Guía oficial no disponible.

Object Management Group. (2017). *OMG Unified Modeling Language Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1/

Python Software Foundation. (s. f.). *abc — Abstract Base Classes*. Python 3.12 documentation. https://docs.python.org/3.12/library/abc.html